

T07 플랜두씨 다이어리 2

로그인 없이 열리던 Plan-Do-See 다이어리에 Supabase Auth를 붙이고, 서버(RLS)가 “이 자료는 이 계정 것”을 강제하게 만든 기록 · 2026-09-05

결과물

whiteclover0542.github.io/plan_record/
첫 화면이 로그인 화면이다.

소스 (고정 COMMIT)

github.com/whiteclover0542/plan_record/tree/6b6673b
T06 이력이 조상 커밋으로 포함(a1a7c29).

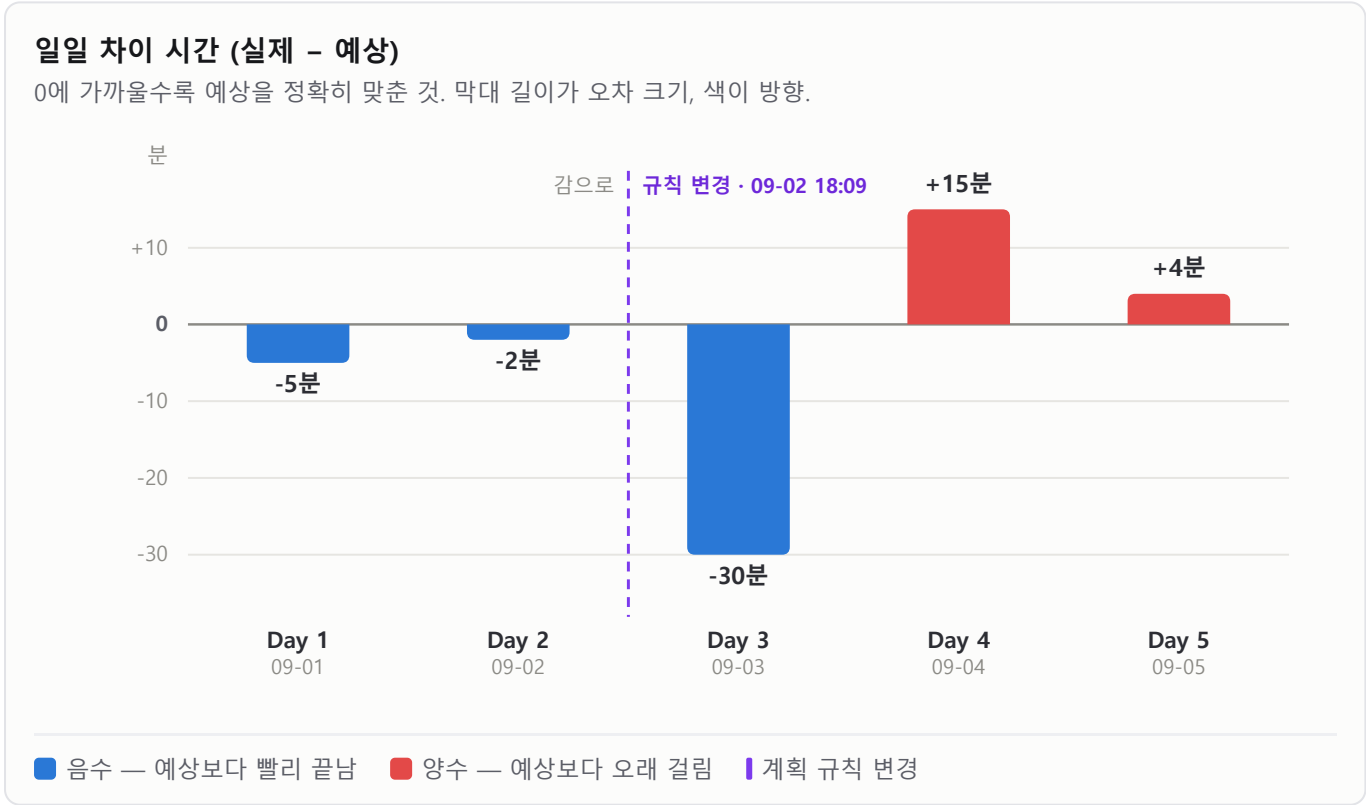
카드 5 · 잠근 앱으로 5일 써 보기

로그인이 걸린 앱을 실제 내 계정으로 서로 다른 실제 날짜 5일 동안 썼고, 2일차 뒤·3일차 앞에 계획 규칙을 한 번 바꿨다.

1일차에 먼저 고정한 것

질문: 계획한 우선순위·시간 배분대로 하루 할 일을 빠짐없이 소화하고 있는가? 지표: 그날 완료한 할 일의 일일 차이 시간(실제-예상) 합계 · 단위: 분

계산 규칙: 결측(미완료)은 제외 · 중복(실행기록 2건 이상)은 최근 1건만 · 이상치(실제가 예상의 5배 이상이거나 음수)는 제외하고 메모 · 분 단위 반올림 · 주 시작은 월요일



일자	날짜	완료	예상(분)	실제(분)	일일 차이(분)	규칙
Day 1	2026-09-01	1	60	55	-5	변경 전(감으로)
Day 2	2026-09-02	2	130	128	-2	변경 전(감으로)
계획 규칙 변경 · 2026-09-02 18:09 (Asia/Seoul) — Day 2 기록 직후, Day 3 시작 전 · 예상 시간을 "감으로" → "비슷한 예전 할 일의 실제 걸린 시간을 참고해서"						
Day 3	2026-09-03	1	120	90	-30	변경 후(적용 안 함)
Day 4	2026-09-04	1	90	105	+15	변경 후(적용함)
Day 5	2026-09-05	1	60	64	+4	변경 후(적용함)
합계		6	460	442	-18	

규칙 변경 전 vs 후 — 절대 오차 평균

변경 전 Day 1~2 3.50 분	변경 후 Day 3~5 16.33 분	새 규칙을 실제로 적용한 날만 Day 4~5 9.50 분
----------------------------------	-----------------------------------	--

부호와 무관하게 “예상을 얼마나 잘 맞췄는가”를 보려면 절대 오차로 봐야 한다. 전후 비교는 같은 지표·같은 단위·같은 계산 규칙으로만 했다.

결론 — 예상 시간을 “감으로 적기”에서 “비슷한 과거 실제 기록을 보고 적기”로 바꿨지만, 절대 오차는 3.50분 → 16.33분(실제 적용한 이틀만 봐도 9.50분)으로 **줄지 않고 오히려 커졌다**. 다만 규칙을 바꾼 시점과 작업 유형이 반복형 집안일에서 개발 작업으로 바뀐 시점이 겹쳐 있어, 이 5일만으로는 규칙의 효과와 작업 유형의 효과를 분리할 수 없다. 같은 유형의 작업으로 며칠 더 기록해야 규칙 효과를 말할 수 있다. 보조 관찰: 5일 내내 지연 0건·막힘 0건, high 우선순위 할 일은 2번 나왔고 둘 다 완료(2/2).

화면에 보이는 합계와 손으로 더한 값 대조

구분	예상(분)	실제(분)	차이(분)
화면 — 집정리하기	190	183	-7
화면 — CVE_정보판 사이트 이어서 개발하기	270	319	+49
화면 합계	460	502	+42
손 계산 합계 (위 5일 표)	460	442	-18
일치 여부	일치	60분 차이	60분 차이

60분 차이의 원인을 규명했다 — Day 5 할 일에 실행기록이 2건(60분·64분) 쌓였는데, 화면은 실행기록을 전부 더하고(`Aggregation.computeForPlan`), 지표는 1일차에 고정된 중복 규칙대로 최근 1건(64분)만 쓴다. 차액 60분이 중복된 옛 기록 1건과 정확히 일치한다. `execution_logs`에는 DELETE 정책이 없어(append-only) 옛 기록을 지워 화면을 맞출 수 없고, 그렇게 하는 대신 **미리 정해 둔 규칙을 지키고 차이를 근거로 남겼다**. (중복 경위: 완료 품의 “실제 걸린 시간” 칸 기본값이 예상 시간이라 60분이 저장됨 → 되돌리기 후 64분으로 재완료 → `complete_todo` RPC가 완료할 때마다 새로 INSERT.)

대조할 때 함께 확인해야 할 것: ① **미완료 할 일** — 화면의 계획별 차이는 미완료 할 일의 예상 시간까지 더한다. 실제로 Day 5 할 일을 완료하기 전에는 이 계획의 차이가 -75분으로 보였고 지표와 달랐다. ② **중복 실행기록** — 위 사례. "T06 플랜두씨 다 이어리 앱 만들기" 계획은 이번 5일 로그 대상이 아니라 제외했다.

인증 구현 설명서

① 무엇으로 붙였나

Supabase Auth(이메일+비밀번호), `@supabase/supabase-js@2.112.4`.

② 왜 그걸 골랐나

이미 Supabase Postgres + Row Level Security 위에서 이 앱이 동작 중이라 `auth.uid()` 로 바로 연결된다. 검토했으나 고르지 않은 방법:

- **직접 구현(bcrypt+JWT)** — 세션 무효화·토큰 재발급 등 보안 디테일을 직접 구현·검증하는 부담이 큼
- **별도 인증 서비스(Auth0/Clerk)** — 인프라가 하나 더 늘고 DB RLS와의 연동 지점이 늘어남

③ 어디를 어떻게 고쳤나

- **가입:** `#signup-form` → `pds-auth.js` → `pds-app.js` (`Auth.signUp`) → `POST /auth/v1/signup` → `auth.users` 에 계정 생성(비밀번호는 bcrypt 해시로 저장)
- **로그인:** `#login-form` → `pds-auth.js` → `Auth.signIn` → `POST /auth/v1/token?grant_type=password` → JWT access/refresh token 발급
- **로그아웃:** `#logout-btn` → `Auth.signOut` → `POST /auth/v1/logout` → 세션 사라짐을 감지해 로그인 화면으로 복귀
- **자료 조회:** `pds-ui.js` 의 `refreshPlans()` → `Plans.list` → PostgREST가 요청 JWT의 `auth.uid()` 로 `plans_select` 정책(`using (user_id = auth.uid())`)을 적용해 내 자료만 반환

DB 쪽: `supabase/migration-t07-auth.sql` 로 기존 T06 데이터를 보존한 채 5개 표

(`plans` / `plan_revisions` / `todos` / `execution_logs` / `retrospective_notes`) 전부에 `user_id` 컬럼을 추가하고 RLS를 `authenticated` + `user_id = auth.uid()` 기준으로 재작성했다. `anon` 룰에는 이 5개 표 어떤 권한도 남기지 않았다.

④ 안 열리는 것을 확인한 기록

`anon` key만으로 라이브 프로젝트에 curl로 직접 요청을 보내 확인(일회용 테스트 계정 사용, 비밀번호는 어디에도 남기지 않음). 아래는 실제 관측된 상태 코드다.

확인	성공 요청	거절 / 무효 요청
가입 중복	새 이메일 signup → 200	같은 이메일 재가입 → 422 user_already_exists
로그인 문구	올바른 비밀번호 → 200	틀린 비밀번호 / 없는 이메일 → 둘 다 동일하게 400 Invalid login credentials
비밀번호 저장	—	encrypted_password = \$2a\$10\$... (bcrypt). 같은 비밀번호로 만든 두 계정도 서로 다른 값
세션	로그인 상태 GET /rest/v1/plans → 200	로그아웃 뒤 refresh token 재사용 → 400 refresh_token_not_found
계정 간 읽기·수정·삭제	본인 자료 → 200 (정상 반영)	남의 행 양방향 시도 → 200 + 빈 배열(값 변경 없음, 실제 대조 확인). 소유권 위조·교차 INSERT → 403 42501
주소 위조	—	?user_id=eq.<남의 uuid> → 200 + 빈 배열(RLS가 서버에서 강제)
비로그인 직접 요청	—	Authorization 없이 요청 → 401 permission denied for table plans
계정 삭제	delete_my_account() → 204	삭제 후 재로그인 → 400 invalid_credentials, 자료도 FK cascade로 실제 삭제

⑤ AI와 나

인증 방식은 AI가 Supabase Auth를 추천했고 내가 최종 승인했다. RLS 마이그레이션 SQL과 로그인 게이트 코드는 AI가 작성했지만, 대시보드에서 단계별로 실제 실행한 것과 실제점 생성·데이터 이관 확인은 내가 직접 했다. 카드 2~4의 거절 근거는 AI가 라이브 프로젝트에 curl로 요청을 보내 모았고, 예상과 다른 관측이 나왔을 때 문장을 고쳐 덮는 대신 원인을 규명해 ⑥에 그대로 남기게 했다. 5일 로그의 질문·지표·계산 규칙은 내가 정했고 기록도 직접 입력했다.

⑥ 아직 못 막은 것

- **로그아웃·비밀번호 변경·계정 삭제 중 무엇을 해도, 그 전에 발급된 access token(JWT)은 자연 만료(최대 1 시간) 전까지 계속 통한다.** refresh token과 계정 자체는 즉시 무효화되지만, access token은 서명·만료만으로 자체 검증되는 무상태 토큰이라 서버가 개별 취소를 기억하지 않기 때문이다. 토큰이 탈취되면 최대 1시간의 위험 구간이 남는다. 완화책은 짧은 만료(1시간)뿐이고, 서버 쪽 토큰 블랙리스트나 세션 버전 관리는 붙이지 않았다.
- **남의 행 수정·삭제 시도가 항상 명시적 403은 아니다** — 경우에 따라 “조용한 무효 처리”(200 + 빈 배열)로 응답한다. 원인: Postgres RLS는 UPDATE/DELETE 대상 행을 찾을 때 그 표의 SELECT 정책도 함께 적용하므로, 애초에 SELECT를 통과 못 하는 남의 행은 대상 후보로도 안 보인다. 데이터는 안전하지만 응답 형태가 균일하지 않다.
- **무차별 대입(brute-force) 로그인 시도 횟수 제한을 이 앱이 직접 만들지 않았다** — Supabase Auth 자체의 기본 rate limit에 의존한다.
- **비밀번호 재설정 메일의 링크 클릭은 실제로 봐보지 못했다** — 기능·화면 전환은 확인했으나 메일함 접근이 안 됐다.

카드 1~4 · 증거 요약

카드 1 — 무엇으로 붙일지 고르고, 이유를 적기

Supabase Auth(이메일+비밀번호), @supabase/supabase-js@2.112.4 . 고르지 않은 방법과 이유는 설명서 ②. 가입 200 · 로그인 200 · 로그아웃 204 · 중복 가입 422 · 로그인 실패 문구는 두 경우 완전히 동일. 로그인 없이 자료 화면 주소를 열면 로그인 화면만 보이고, 서버도 401 로 거절한다. T06 자료 이관은 still_null=0 (계획 5건 전부)로 확인했고, 배포 사이트에서 UI 클릭으로 14개 항목을 재확인했다.

카드 2 — 비밀번호를 어떻게 맡아 두는지 보이기

저장은 전부 Supabase Auth(GoTrue)에 위임했고 앱 코드는 비밀번호를 직접 해싱·저장하지 않는다. 실제 저장 값은 \$2a\$10\$... — bcrypt(cost 10)이고 입력한 글자는 어디에도 보이지 않는다. 같은 비밀번호로 만든 두 계정의 저장 값이 서로 다를 수 대조로 확인했다(salt가 다르기 때문). 수집한 모든 요청/응답을 실제 비밀번호 문자열로 grep해 원문이 전무함을 확인했다.

카드 3 — 들어온 사람을 어떻게 기억하는지 보이기

JWT access token(alg: ES256, 비대칭키) + refresh token. 만료는 expires_in=3600 (1시간)으로, JWT payload의 exp / iat 를 직접 디코드해 확인했다. 토큰은 URL에 실리지 않고 Authorization 헤더로만 전달된다. 같은 주소·같은 방식으로 로그아웃 전/후를 비교했고, refresh token은 즉시 거절되지만 access token은 만료 전까지 통한다는 점을 ⑥에 적었다. 이 프로젝트는 ES256 이라 공유 비밀키 자체가 없는 구조이며, 저장소 전체 grep으로 클라이언트 코드·배포 파일·Git 이력에 비밀키가 없음을 확인했다.

카드 4 — 남의 자료가 안 열리는 것을 보이기

계정 두 개를 만들어 각각 자료를 넣고 양방향으로 교차 접근을 시도했다. 비밀번호는 이 문서 어디에도 적지 않았다. 읽기·수정·삭제는 양방향 모두 무효(200 + 빈 배열, 상대 자료의 제목·건수 실제 대조로 변경 없음 확인), 소유권 위조와 교차 INSERT는 403 42501, 주소에 남의 계정 id를 필터로 넣어도 빈 배열, complete_todo RPC로 남의 할 일을 완료 처리하려 해도 newly_completed: false, 비로그인 직접 요청은 401. 목록에는 본인 자료만 나온다. 거절을 만드는 소스는 migration-t07-auth.sql STEP 6(RLS 정책)·STEP 8(RPC 소유자 조건)이다.

읽기·수정·삭제가 설계 의도였던 403 이 아니라 200 + 빈 배열로 막히는 이유까지 SQL로 규명해 ⑥에 적었다 — 응답 코드를 의도대로 꾸미지 않고 실제 관측값을 그대로 남겼다.

짧은 확인 방법

1. 어디로 가나요 — whiteclover0542.github.io/plan_record/ 접속(첫 화면이 로그인/가입 화면).
2. 세 단계 안에 무엇을 하나요 — (1) 가입하기로 새 계정을 만들거나 기존 계정으로 로그인 (2) “1. 계획” 탭에서 계획 목록을 본다 (3) “3. 돌아보기” 탭에서 집계 숫자를 확인한다.
3. 무엇이 보이면 통과인가요 — 로그인 전에는 로그인 화면만 보이고, 로그인 후에는 그 계정에 속한 자료만 보인다(다른 계정 자료는 절대 섞이지 않는다).
4. 안 될 때는 무엇이 보이나요 — 로그인 없이 자료 화면 주소를 열면 계속 로그인 화면만 보이고, 개발자 콘솔에는 Supabase REST 요청이 401/403으로 거절된 로그가 찍힌다.

AI와 내 판단 3줄

- ① **AI에게 맡긴 일** — RLS 소유권 마이그레이션 SQL과 로그인 게이트 코드 작성, curl로 카드 2~4의 거절 근거 수집, 예상과 다른 관측이 나왔을 때 원인 규명, 문서 정리.
- ② **내가 판단한 일** — 인증 방식 최종 승인, 마이그레이션을 대시보드에서 직접 단계별 실행, 5일 로그의 질문·지표·계산 규칙 확정과 실제 기록 입력.
- ③ **AI 제안을 따르지 않은 일** — 중복된 실행기록을 지워 화면 숫자를 맞추자는 제안 대신, 1일차에 정해 둔 규칙(중복은 최근 1건)을 지키고 60분 차이를 근거로 남겼다. 미완료 할 일을 억지로 완료 처리하지 않은 것도 같은 이유다.

제출물 확인

- ✓ 가입·로그인·로그아웃이 되고, 로그인 없이 자료 화면을 열면 로그인 화면이 나옵니다.
- ✓ 저장된 비밀번호에 입력한 글자가 보이지 않습니다.
- ✓ 로그아웃 뒤 같은 값으로 다시 요청하면 거절됩니다. — refresh token은 즉시 거절. access token은 만료 전까지 통하며, 이 한계를 ⑥에 적었습니다.
- ✓ 남의 자료를 읽기·수정·삭제하려는 요청이 양방향으로 모두 거절되고, 목록에도 섞이지 않습니다.
- ✓ 설명서 여섯 항목이 나뉘어 있고, ⑥이 비어 있지 않습니다.
- ✓ 서로 다른 날짜 5일의 기록과 3일차 전 규칙 변경이 순서대로 있습니다.
- ✓ 제출물에 비밀번호·토큰·비밀키 원문이 없습니다. — 저장소 전체 grep으로 확인. `config.js`의 anon key 하나뿐이며, 이는 Supabase 설계상 공개되는 키입니다.